

WFLOW-99: Best Practice Summary

Series: WFLOW — Workflows and Alert Notifications | Notebook: 99 | Created: March 2026 | Last Updated: 09/28/2026

Overview

This notebook consolidates every actionable best practice from the WFLOW series (notebooks 01-09) into definitive tables organized by category. Each best practice includes the exact setting or value to use, a priority rating, and the source notebook.

Table of Contents

- 1. [Workflow Design & Architecture](#)
- 2. [Triggers](#)
- 3. [Connections & Credentials](#)
- 4. [Notification Channels](#)
- 5. [Message Formatting & Templates](#)
- 6. [Routing & Escalation](#)
- 7. [Incident Management Integration](#)
- 8. [Auto-Remediation](#)
- 9. [JavaScript & HTTP Actions](#)
- 10. [Security](#)
- 11. [Access Control](#)
- 12. [Observability & Monitoring](#)
- 13. [Operations & Change Management](#)

1. Workflow Design & Architecture

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use Workflows for all new automation	Workflows (not legacy alerting profiles)	Critical	WFLOW-01

#	Best Practice	Recommended Setting/Value	Priority	Source
2	Keep workflows small and focused	One trigger → one outcome; avoid sprawling multi-stage workflows (community practice — explicit max-execution-time is not published in current docs; verify against your tenant under load)	Recommended	WFLOW-01
3	Keep task count modest	~20 tasks per workflow as a soft ceiling (community practice — split larger flows into sub-workflows). Hard max not published in current docs.	Recommended	WFLOW-01
4	Set explicit task timeouts	Default is 60 minutes; max 7 days. Configure <code>timeout: <seconds></code> per task. Distinct from the 120s Dynatrace runtime budget that caps each action (DQL/JS) inside the task.	Critical	WFLOW-01, WFLOW-08
5	Avoid uncontrolled fan-out	Throttle high-cardinality triggers (community practice — explicit concurrent-execution cap is not published in current docs; verify against your tenant).	Recommended	WFLOW-01
6	Start with simple notifications	Basic notifications first, then layer in complexity	Recommended	WFLOW-09
7	Run notification tasks in parallel	Default parallel execution for independent tasks (Slack + Teams + Email simultaneously)	Recommended	WFLOW-03
8	Use sequential execution with <code>dependsOn</code>	Chain tasks that require previous task output	Recommended	WFLOW-04

2. Triggers

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use Detected Problem trigger for incident alerts	<code>type: davis-problem</code>	Critical	WFLOW-02
2	Use the Davis event trigger only for per-alert reactions	Put the threshold in an anomaly detector (ALERT-02); trigger on the problem, and use the Davis event trigger only when you need per-alert granularity	Critical	WFLOW-02
3	Use Schedule trigger for reports and health checks	<code>type: schedule</code> with cron expression and explicit <code>timezone</code>	Critical	WFLOW-02
4	Use On-Demand trigger for testing	<code>type: on-demand</code> before deploying any production workflow	Critical	WFLOW-02

#	Best Practice	Recommended Setting/Value	Priority	Source
5	Scope Detected Problem triggers with entity tags	<code>entityTagsMatch: all with entityTags: [{key: env, value: prod}]</code>	Critical	WFLOW-02
6	Filter by problem categories	Set <code>categories: [AVAILABILITY, PERFORMANCE]</code> to reduce noise	Recommended	WFLOW-02
7	Handle all problem lifecycle events	Problem state <i>active or closed</i> ; branch on <code>event.status (ACTIVE / CLOSED)</code> in a single workflow	Recommended	WFLOW-05
8	Offset scheduled workflows from minute :00	Use <code>:05</code> , <code>:15</code> , <code>:30</code> instead of <code>:00</code> to spread load	Recommended	WFLOW-02
9	Always set timezone on schedule triggers	<code>timezone: "America/New_York"</code> (explicit, never rely on default)	Recommended	WFLOW-02
10	Use Event trigger for business process automation	<code>type: event with filterQuery</code> for bizevent-driven workflows	Optional	WFLOW-02

3. Connections & Credentials

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use descriptive connection names	Pattern: <code><service>-<environment>-<purpose></code> (e.g., <code>slack-prod-oncall</code>)	Critical	WFLOW-03
2	Separate connections per environment	Distinct connections for prod vs staging vs dev	Critical	WFLOW-03
3	Apply least-privilege scopes	Minimum OAuth scopes required (e.g., Slack: <code>chat:write</code> , <code>chat:write.public</code> only)	Critical	WFLOW-03
4	Rotate tokens on a schedule	Monthly rotation of API tokens and passwords	Recommended	WFLOW-09
5	Use OAuth 2.0 over Basic Auth for ServiceNow	<code>OAuth 2.0</code> for production; Basic Auth only for dev/testing	Recommended	WFLOW-05
6	Create separate PagerDuty connections per service routing	<code>pagerduty-prod-critical</code> , <code>pagerduty-prod-standard</code> , <code>pagerduty-platform</code>	Recommended	WFLOW-05
7	Prefer Slack OAuth App over Incoming Webhook	OAuth App provides channels, DMs, reactions, threads	Recommended	WFLOW-03

4. Notification Channels

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Multi-channel for Critical production problems	PagerDuty + Slack #urgent + Email simultaneously	Critical	WFLOW-03, WFLOW-04
2	Slack for High production problems	Slack #alerts + Email	Recommended	WFLOW-04
3	Slack-only for Medium problems	Slack #alerts (business hours only for Low)	Recommended	WFLOW-04
4	Use Power Automate connector for Teams	Microsoft is deprecating Office 365 Connectors; use newer Workflows connector	Recommended	WFLOW-03
5	Use built-in email for simple notifications	No SMTP setup required; use custom SMTP or SendGrid for high volume	Optional	WFLOW-03
6	Include severity in email subject line	<pre>[[{ event() ['event.category'] }]] {{ event() ['event.name'] }}</pre>	Recommended	WFLOW-03

5. Message Formatting & Templates

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Structure messages: Severity + Title + Key metrics + Link	Line 1: severity indicator + title; Line 2: impact; Line 3: affected entities; Line 4: action link	Critical	WFLOW-06
2	Use severity-coded visual indicators	Slack, keyed on <code>event.severity</code> : <code>:red_circle:</code> 1, <code>:large_orange_circle:</code> 2, <code>:large_yellow_circle:</code> 3, <code>:large_blue_circle:</code> 4	Critical	WFLOW-06
3	Use Slack Block Kit for rich messages	<code>blocks:</code> with <code>header</code> , <code>section</code> , <code>actions</code> , <code>context</code> types	Recommended	WFLOW-06
4	Use Teams Adaptive Cards v1.4	<code>type:</code> <code>AdaptiveCard</code> , <code>version:</code> "1.4" with <code>FactSet</code> , <code>TextBlock</code> , <code>Action.OpenUrl</code>	Recommended	WFLOW-06
5	Always include a link to the Dynatrace problem	<code><{{ problem_link() }} View in Dynatrace></code> (Slack) or <code>Action.OpenUrl</code> (Teams)	Critical	WFLOW-03

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Use dictionary mapping for severity-to-emoji	<code>{{ {1: ":red_circle:", ...}.get(event().get("event.severity") \ \nint(5), ":white_circle:") }}</code>	Recommended	WFLOW-06
7	Limit affected entities shown to 3-5	<code>event()["affected_entity_ids"][:3]</code> with overflow indicator	Recommended	WFLOW-06
8	Use <code>.get()</code> with defaults for optional fields	<code>event().get("root_cause_entity_id", "Pending analysis")</code>	Critical	WFLOW-06
9	Truncate long strings	<code>{{ event()["event.name"] \ \ntruncate(50) }}</code>	Recommended	WFLOW-06
10	Enrich notifications with DQL data	Add recent error logs via <code>queryExecutionClient.queryExecute()</code> in a JavaScript task before the notification task	Optional	WFLOW-06
11	Test templates with mock data first	On-Demand trigger + JavaScript mock event task before production deployment	Critical	WFLOW-06
12	Preview Slack templates in Block Kit Builder	https://app.slack.com/block-kit-builder before deploying	Recommended	WFLOW-06
13	Create a resolved-problem template	<code>:white_check_mark:</code> Problem Resolved with duration and resolution time	Recommended	WFLOW-06

6. Routing & Escalation

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Route by severity	<code>event.severity</code> 1: PagerDuty + Slack + Email; 2: Slack + Email; 3: Slack; 4: Slack business hours only. Check which sources set severity: SaaS 1.348 stops defaulting it to 3 (pre-release, staged rollout)	Critical	WFLOW-04
2	Route by team ownership via entity tags	Condition: <code>"team:checkout"</code> in <code>event().get("tags", [])</code> maps to <code>#checkout-alerts</code>	Critical	WFLOW-04

#	Best Practice	Recommended Setting/Value	Priority	Source
3	Do not route by management zone	<code>"Production" in event()</code> <code>["management_zones"]</code> is legacy. It fails silently once MZs are retired — the array empties, conditions go false, notifications stop with no error. Route on entity tags (<code>env:prod</code>) or Smartscape ownership instead	Critical	WFLOW-04
4	Implement time-based routing	Business hours (Mon-Fri 9-17): Slack channel; Off-hours Critical: PagerDuty; Off-hours non-critical: queue for morning	Critical	WFLOW-04
5	Auto-escalate unacknowledged alerts	Wait 15 min, re-query the problem and check it is still ACTIVE, escalate to PagerDuty if unacknowledged	Recommended	WFLOW-04
6	Multi-tier escalation	0 min: Slack; 15 min: Email team lead; 30 min: PagerDuty on-call; 60 min: PagerDuty manager	Recommended	WFLOW-04
7	Use AND logic for multi-condition tasks	<code>conditions: [is_critical, is_production]</code> requires both true	Recommended	WFLOW-04
8	Use JavaScript for dynamic channel selection	Parse <code>team:</code> tag to construct <code>#{team}-alerts</code> dynamically	Optional	WFLOW-04
9	Separate environment routing	Production: immediate notification; Staging: Slack only; Dev: daily digest	Recommended	WFLOW-04

7. Incident Management Integration

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Deduplicate PagerDuty incidents	<code>dedupKey: "dynatrace-{{ event() ['display_id'] }}"</code>	Critical	WFLOW-05
2	Deduplicate ServiceNow incidents	<code>correlation_id: "DT-{{ event() ['display_id'] }}"</code>	Critical	WFLOW-05
3	Map Dynatrace severity to PagerDuty severity	<code>event.severity 1: critical ; 2: error ; 3: warning ; 4: info</code>	Critical	WFLOW-05
4	Map Dynatrace severity to ServiceNow impact/urgency	<code>event.severity 1–2: impact=1, urgency=1; 3: impact=2, urgency=2; 4: impact=3, urgency=3</code>	Critical	WFLOW-05
5	Auto-resolve incidents when problem closes	Trigger on <code>event()["event.status"] == "CLOSED"</code> , resolve PagerDuty via <code>dedupKey</code> or set ServiceNow <code>state: 6</code>	Critical	WFLOW-05

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Include Dynatrace problem URL in incident details	<code>{{ problem_link() }}</code> in <code>customDetails</code> (PagerDuty) or the <code>description</code> body (ServiceNow)	Critical	WFLOW-05
7	Store incident ID for subsequent updates	Capture <code>sys_id</code> from create response in a JavaScript task for update/resolve operations	Recommended	WFLOW-05
8	Use PagerDuty Events API v2	Integration type: Events API v2 (not v1)	Critical	WFLOW-05
9	Set <code>assignment_group</code> and <code>caller_id</code> in ServiceNow	<code>assignment_group</code> : "Platform Engineering", <code>caller_id</code> : "dynatrace.integration"	Recommended	WFLOW-05
10	Implement bi-directional sync	Scheduled workflow queries ServiceNow for open DT-correlated incidents and updates Dynatrace problem comments	Optional	WFLOW-05

8. Auto-Remediation

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Rate-limit remediation attempts	Max 3 remediations per hour per entity	Critical	WFLOW-07
2	Enforce time-window guardrails	Remediation only during safe hours: weekdays 06:00-22:00	Critical	WFLOW-07
3	Implement cooldown periods	30 minutes minimum between remediation actions on the same entity	Critical	WFLOW-07
4	Capture pre-remediation state for rollback	Record current state before executing any change	Critical	WFLOW-07
5	Require human approval for production remediation	Slack approval buttons with 30-minute timeout; auto-approve for non-production only	Critical	WFLOW-07
6	Auto-remediate only well-defined scenarios	Pod crash loops: yes (restart); Disk space 90%: yes (cleanup); Database deadlocks: no (investigate); Security incidents: no (human judgment)	Critical	WFLOW-07
7	Follow remediation maturity model	Level 0: manual; Level 1: notification + runbook link; Level 2: semi-automated (approval); Level 3: fully automated with guardrails	Recommended	WFLOW-07

#	Best Practice	Recommended Setting/Value	Priority	Source
8	Map problem types to runbooks	JavaScript <code>runbookMap</code> object mapping problem title patterns to runbook IDs	Recommended	WFLOW-07
9	Include runbook links in notifications	Jinja conditional: <code>{% if "CPU" in event()["event.name"] %}</code> links to CPU runbook	Recommended	WFLOW-07
10	Validate after remediation	Query metrics/logs post-action to confirm resolution	Recommended	WFLOW-07
11	Start non-prod, promote to prod	Test all remediation workflows in staging before enabling in production	Critical	WFLOW-07

9. JavaScript & HTTP Actions

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Always use <code>export default async function</code>	Required function signature for every JavaScript action	Critical	WFLOW-08
2	Wrap all external calls in try-catch	Return <code>{success: false, error: error.message}</code> on failure instead of throwing	Critical	WFLOW-08
3	Implement retry with exponential backoff	Max 3 retries, delay = <code>1000 * attempt</code> ms, retry only on 5xx errors	Recommended	WFLOW-08
4	Set request timeouts on external HTTP	<code>AbortController</code> with 10-second timeout on outbound <code>fetch()</code> calls. Distinct from task <code>timeout</code> (whole-task budget) and 120s runtime budget (per-action). See WFLOW-08 §8.	Critical	WFLOW-08
5	Poll DQL queries to completion	<code>queryExecute()</code> returns the result only if the query finishes within <code>requestTimeoutMilliseconds</code> ; otherwise it returns a <code>requestToken</code> . Poll with <code>queryPoll()</code> until the state is final, and fail on anything but <code>SUCCEEDED</code> — never read <code>result.result.records</code> unchecked. The 120s runtime budget still applies. See WFLOW-08 §2 and §10.	Critical	WFLOW-08
6	Limit DQL query scope	<code>from: now() - 1h</code> , select only needed <code>fields</code> , <code>limit 100</code>	Critical	WFLOW-08
7	Use <code>Promise.all()</code> for parallel entity lookups	Execute all independent API calls concurrently	Recommended	WFLOW-08

#	Best Practice	Recommended Setting/Value	Priority	Source
8	Always use HTTPS for external calls	Never use <code>http://</code> in HTTP request URLs	Critical	WFLOW-09
9	Read secrets from the Credential Vault	<code>credentialVaultClient.getCredentialsDetails({ id })</code> in JavaScript; the Authentication field in HTTP tasks (never hardcode, never log or return secrets)	Critical	WFLOW-08, WFLOW-09
10	Add problem comments after remediation	Use <code>problemsClient.createComment()</code> to record automated actions	Recommended	WFLOW-08
11	Validate secret existence before use	<code>if (!apiToken) throw new Error('Missing required secret: ...')</code>	Recommended	WFLOW-09

10. Security

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Never hardcode secrets in workflows	Connections for connector actions; Credential Vault for HTTP (Authentication field) and JavaScript (<code>credentialVaultClient</code>)	Critical	WFLOW-09
2	Sanitize all dynamic inputs	Remove <code>"'\\" characters, limit string length to 200 chars before using in queries</code>	Critical	WFLOW-09
3	Never log secrets	Do not <code>console.log()</code> a credential or return it in task output — results are visible to anyone who can read the workflow	Critical	WFLOW-09
4	Name credentials consistently	Pattern: <code><service>-<env>-<purpose></code> for Credential Vault entries and connections	Recommended	WFLOW-09
5	Rotate secrets monthly	Update the Credential Vault entry or connection, test, revoke the old token at its source	Recommended	WFLOW-09
6	Fail secure on errors	Default to safe state (no action) when errors occur, never expose error details externally	Critical	WFLOW-09

11. Access Control

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Grant minimum permissions per role	Viewers: <code>automation:workflows:read</code> ; App teams: <code>read + write + run</code> (own workflows); SRE: <code>read + write + run</code> (all); Admins: <code>admin</code>	Critical	WFLOW-09
2	Restrict workflow write to owners	IAM policy: <code>ALLOW automation:workflows:write WHERE workflow.owner == "\${user.email}"</code>	Recommended	WFLOW-09
3	Separate connection write access	<code>automation:connections:write</code> only for workflow admins and SRE	Critical	WFLOW-09
4	Tag workflows with ownership metadata	<code>metadata.owner: sre-team@company.com</code> , <code>metadata.team: platform</code> , <code>metadata.classification: production</code>	Recommended	WFLOW-09

12. Observability & Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Alert on workflow success rate	Threshold: < 95% success rate over 7 days	Critical	WFLOW-09
2	Alert on average execution duration	Threshold: > 5 minutes average	Recommended	WFLOW-09
3	Alert on failure spike	Threshold: > 5 failures per hour	Critical	WFLOW-09
4	Build a workflow health dashboard	Queries: overall health, per-workflow success rates, execution trend, recent failures, task-level performance	Critical	WFLOW-09
5	Create a workflow-monitoring workflow	Scheduled every 15-60 min; query for workflows with 3+ failures/hour; alert to <code>#workflow-alerts</code>	Recommended	WFLOW-09
6	Monitor notification task success rates	Query <code>dt.system.events ACTION_EXECUTION</code> records by <code>dt.automation_engine.action.app</code> (<code>dynatrace.email</code> , <code>dynatrace.slack</code> , ...)	Recommended	WFLOW-03

#	Best Practice	Recommended Setting/Value	Priority	Source
7	Track remediation success rates	Query <code>dt.system.events</code> <code>WORKFLOW_EXECUTION</code> records filtered on <code>dt.automation_engine.workflow.title</code> containing <code>remediation</code>	Recommended	WFLOW-07
8	Monitor skipped tasks	Query <code>TASK_EXECUTION</code> records with <code>dt.automation_engine.state</code> <code>SKIPPED</code> / <code>DISCARDED</code> to validate routing conditions are working as designed	Optional	WFLOW-04

13. Operations & Change Management

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Name workflows consistently	Pattern: <code><team>-<function>-<environment></code> (e.g., <code>sre-problem-notifications-prod</code>)	Critical	WFLOW-09
2	Export workflows as JSON for version control	<code>GET /platform/automation/v1/workflows/<id></code> saved to Git	Recommended	WFLOW-09
3	Test changes on cloned workflow first	Clone, modify, test with On-Demand trigger, review results, promote	Critical	WFLOW-09
4	Rollback procedure	Disable failing workflow (toggle off), import previous JSON, enable restored version, verify	Critical	WFLOW-09
5	Daily: check execution dashboard	Review success rates and investigate failures	Recommended	WFLOW-09
6	Weekly: verify external connections	Test all Slack, Teams, PagerDuty, ServiceNow connections	Recommended	WFLOW-09
7	Monthly: rotate secrets	Update API tokens, passwords, webhook URLs	Recommended	WFLOW-09
8	Query execution history with DQL	<code>fetch dt.system.events, from:-24h \ filter event.kind == "WORKFLOW_EVENT" and event.type == "WORKFLOW_EXECUTION"</code>	Recommended	WFLOW-01

Summary

This notebook contains **91 best practices** across 13 categories extracted from the complete WFLOW series.

Priority distribution:

Priority	Count	Meaning
Critical	47	Must implement for production readiness
Recommended	38	Strongly advised for operational maturity
Optional	6	Beneficial for advanced use cases

Implementation order:

1. Connections and credentials (WFLOW-03)
2. Basic problem notification workflow with severity routing (WFLOW-03, WFLOW-04)
3. Incident management integration with deduplication (WFLOW-05)
4. Custom templates with severity indicators and links (WFLOW-06)
5. Workflow health monitoring dashboard (WFLOW-09)
6. Auto-remediation with guardrails (WFLOW-07)

References

- [Workflows umbrella \(DT docs\)](#)
- [Workflow triggers \(DT docs\)](#)
- [Workflow actions \(DT docs\)](#)
- [Notification actions umbrella \(DT docs\)](#)
- [HTTP request action \(DT docs\)](#)
- [Run JavaScript action \(DT docs\)](#)
- [Workflow reference / Jinja expressions \(DT docs\)](#)
- [Davis Problems app \(DT docs\)](#)
- [Alerting and notifications umbrella \(DT docs\)](#)
- [Upgrade guide — Alert notification \(DT docs\)](#)
- [Dynatrace Developer Portal \(Dynatrace\)](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.